GoTreeSitter

How we built an ambitious parser foundation with AI—and kept it honest

Oscar Villavicencio · M31 Labs · GopherCon 2026

Hi, I'm Oscar

I build developer tools at M31 Labs. GoTreeSitter, GoSX, Markdown++, Canopy, and Graft all ship from one bet: **own the syntax layer once, in pure Go, and let every product start above it.**

This deck is one of those products. Let's look at the layer underneath it.

The next twenty-five minutes

I · WHAT IT IS

An application-ready parsing layer in pure Go—and the C reference implementation it answers to.

II · HOW WE BUILT IT

An AI-assisted loop where evidence, not the model, decides what merges.

III · HOW IT'S USED

Products above the boundary, running live on this deck—and a playbook you can steal.

First, the reference: Tree-sitter

THE C RUNTIME

Over a decade of parser engineering: incremental, error-tolerant, and hardened inside the editors millions of developers type into daily.

THE GRAMMAR ECOSYSTEM

Hundreds of community grammars—accumulated years of real-world edge cases nobody wants to rediscover.

THE GO PROBLEM

Every Go binding wraps the C library through CGo—painful to cross-compile, ship static, or run in WebAssembly.

> **IMPORTANT**
> GoTreeSitter reimplements the runtime in pure Go, keeps the grammar ecosystem, and the C original becomes the reference implementation we test against.

It ships the layer between parsing and product

FIND

Detect a language; fully or incrementally parse it.

READ

Run queries; return tags and typed captures.

PRESENT

Produce highlight ranges and injected child trees.

CHANGE

Apply atomic rewrites; emit the next `InputEdit` records.

> **IMPORTANT**
> Application-ready Go APIs: 206 embedded grammars, 119 hand-written Go scanners, 156 highlight and 69 tags query packs—not a checklist to rebuild.

Start with a file; ask for the capability

APPLICATION CODE

```
entry := grammars.DetectLanguage("handler.ts")
if entry == nil {
    return ErrUnsupported
}

tree, _ := gotreesitter.NewParser(
    entry.Language(),
).Parse(source)
```

> **NOTE**
> GoTreeSitter is more than a parser—and deliberately less than a language server.

WHAT THE ENTRY CAN CARRY

Grammar	portable parser tables for the runtime
Feature queries	highlights and tags when the grammar ships them
Runtime facts	extensions, scanner support, and quality classification

Queries and rewrites are ordinary Go

ASK A STRUCTURAL QUESTION

```
q, _ := gotreesitter.NewQuery(
    `(function_declaration
        name: (identifier) @fn)`, lang)
cur := q.Exec(tree.RootNode(), lang, src)
for {
    m, ok := cur.NextMatch()
    if !ok { break }
    fmt.Println(m.Captures[0].Node.Text(src))
}
```

REWRITE, THEN REPARSE INCREMENTALLY

```
rw := gotreesitter.NewRewriter(src)
rw.Replace(fnName, []byte("newName"))
rw.Delete(unusedNode)

newSrc, _ := rw.ApplyToTree(tree)
newTree, _ := parser.ParseIncremental(
    newSrc, tree)
```

A no-edit reparse returns in single-digit nanoseconds, with zero allocations.

We ported observable behavior—not source code

REFERENCE LANE

Same source

Same grammar version

Tree-sitter C runtime

Normalized structural result

CANDIDATE LANE

Same source

Same grammar version

Pure-Go runtime

Normalized structural result

> **The oracle was the breakthrough**
> Compare node types, child shape, fields, byte ranges, missing nodes, and error placement.

The bet was much larger than “rewrite C in Go”

REPLACE A RUNTIME

Own lexing, parsing, error recovery, scanners, queries, and incremental reuse in Go.

KEEP THE ECOSYSTEM

Load mature Tree-sitter grammars instead of asking every language to start over.

PRESERVE BEHAVIOR

Match observable trees, fields, ranges, errors, and scanner-dependent results.

SHIP THE NEXT LAYER

Expose the application capabilities those structures make possible.

> **IMPORTANT**
> The project was too large to “vibe-check.” Every boundary needed a witness.

AI proposed; evidence decided

PROPOSE

AI explores candidate code, tests, and explanations.

COMPARE

The oracle and invariants reject plausible lies.

REDUCE

One mismatch becomes the smallest source that still fails.

KEEP

The witness becomes a permanent regression test.

> **IMPORTANT**
> The model accelerated the search. It was never the evidence.

Every mismatch became a smaller problem

CORPUS
Find a real file where behavior diverges.

DIFF
Name the first structural disagreement.

REDUCE
Delete everything that does not preserve it.

INVARIANT
State the rule the implementation violated.

RATCHET
Keep the minimal case and move the floor forward.

> **TIP**
> Give AI one small, checkable problem—not “make the whole parser correct.”

Build vertical proof slices

FULL PARSE

Grammar → tables → blob → loader → tree → oracle comparison

PRODUCT CAPABILITY

Query pack → captures → highlight, tag, or injection result → fixture

EDIT LOOP

Old tree → edit → incremental candidate → fresh parse comparison → admit or fall back

> **IMPORTANT**
> A component is not done when its unit test passes; it is done when one real path crosses the system.

Every optimization had to earn its way in

CANDIDATE

An old subtree, scanner checkpoint, or fast path might be reusable.

PROOF

Ranges, parser state, fragility, and external state must still agree.

FALLBACK

Uncertain work returns to the slower production path.

MEASURE

Bench gates verify the safe path is still useful.

> **WARNING**
> A slower honest result is better than a fast structural lie.

grammargen made language work reproducible

A GRAMMAR IS REVIEWABLE GO

```
g := NewGrammar("mini_expr")
g.Define("expression", Choice(
    PrecLeft(1, Seq(
        Field("left", Sym("expression")),
        Field("operator", Str("+")),
        Field("right", Sym("expression")))),
    Sym("number")))
g.Test("precedence", "1 + 2 * 3", "")
```

ONE PIPELINE, ONE ARTIFACT

Author Go DSL or resolved upstream `grammar.json`

Normalize both inputs merge into one internal grammar format

Generate lexer tables, parser tables, fields, conflicts, scanner metadata

Ship one portable grammar blob beside optional query packs

Applications load the artifact—never the generator or its toolchain.

Explicit ownership kept the layers honest

RUNTIME OWNS

Parser execution, recovery, ranges, queries, bounded work, and safe reuse.

GRAMMAR OWNS

Tree vocabulary, fields, conflicts, scanner behavior, feature queries, and corpus compatibility.

PRODUCT OWNS

Scope, resolution, semantics, policy, user experience, and migration intent.

> **NOTE**
> A clear boundary lets people—and AI agents—change one layer without pretending to own the others.

The harness had to ratchet, not merely test

PARITY GATE

Compare observable structure against the reference runtime.

CORPUS GATE

Keep real language families and scanner paths in the loop.

BENCH GATE

Catch fake wins. We withdrew our own headline when the harness caught one.

RELEASE RECEIPT

Pin the grammar, corpus, capability, and version behind each claim.

> **IMPORTANT**
> Once a bug is fixed, it can never quietly come back—the bar only moves up.

The foundation paid back across very different products

GoSX	composes maintained Go syntax with native markup, then compiles the combined tree
Markdown++	gives parsing, formatting, linting, LSP, rendering, and these slides one document model
qml-language-server	built by someone outside this project—QML and Qt workspace meaning above GoTreeSitter trees and ranges
Canopy and Graft	build structural code intelligence and entity-aware version control above shared syntax

> **One of these was built outside the project**
> qml-language-server starts at trees and ranges—the strongest evidence the boundary is in the right place.

This deck is the demo

THE ISLAND SOURCE (GoSX, TRIMMED)

```
type ParseTreeProps struct {
    Expr string
}

//gosx:island
func ParseTree(props ParseTreeProps) Node {
    open := signal.New(true)
    toggle := func() { open.Set(!open.Get()) }
    return <button onClick={toggle}>
        {open.Get() ? "▾" : "▸"}
        function_declaration
    </button>
}
```

THE SAME ISLAND, RUNNING HERE

```
total := price * (count + 1)                                  bytes 0–28
source_file
  ▾ function_declaration
    block → statement_list
    short_var_declaration
      expression_list → identifier total
      anonymous token :=
      ▾ expression_list → binary_expression
        identifier price · anonymous *
        ▸ parenthesized_expression
```

Danmuji: tests you can read aloud

THE DSL (.dmj)

```
package cart_test

import "testing"

unit "ShoppingCart.Add" {
    given "an empty cart" {
        cart := NewCart()
        when "adding an item" {
            cart.Add("widget")
            then "count increases" {
                expect cart.Count() == 1
            }
        }
    }
}
```

THE GENERATED `go test` (TRIMMED)

```
func TestShoppingCartAdd(t *testing.T) {
    t.Parallel()
    t.Run("an empty cart", func(t *testing.T) {
        cart := NewCart()
        t.Run("adding an item", func(t *testing.T) {
            cart.Add("widget")
            t.Run("count increases", func(t *testing.T) {
                assert.EqualValues(t, 1, cart.Count())
            })
        })
    })
}
```

Ferrous Wheel: rusty in, boring Go out

THE DSL (.fw)

```
package main

import "os"

enum Status { Active, Suspended(string) }

derive Equal for Status

func loadUser(path string) (string, error) {
    let data = os.ReadFile(path)?
    let mut name = string(data)
    if name == "" {
        name = "anonymous"
    }
    return name, nil
}
```

THE EMITTED GO (TRIMMED)

```
type Status struct {
    tag        int
    suspended0 string
}

func (x Status) Equal(other Status) bool {
    return x == other
}

func loadUser(path string) (string, error) {
    data, _fwTryErr0 := os.ReadFile(path)
    if _fwTryErr0 != nil {
        return *new(string), _fwTryErr0
    }
    name := string(data)
    // … unchanged from the source
}
```

A playbook for your massive project

1 · FOUNDATION

Choose the narrow foundation that several outcomes can share.

2 · CONTRACT

Name observable behavior and ownership before implementation spreads.

3 · WITNESS

Build an independent oracle, simulator, fixture, or invariant.

4 · SEARCH

Let AI explore freely where wrong answers get caught.

5 · COMPOUND

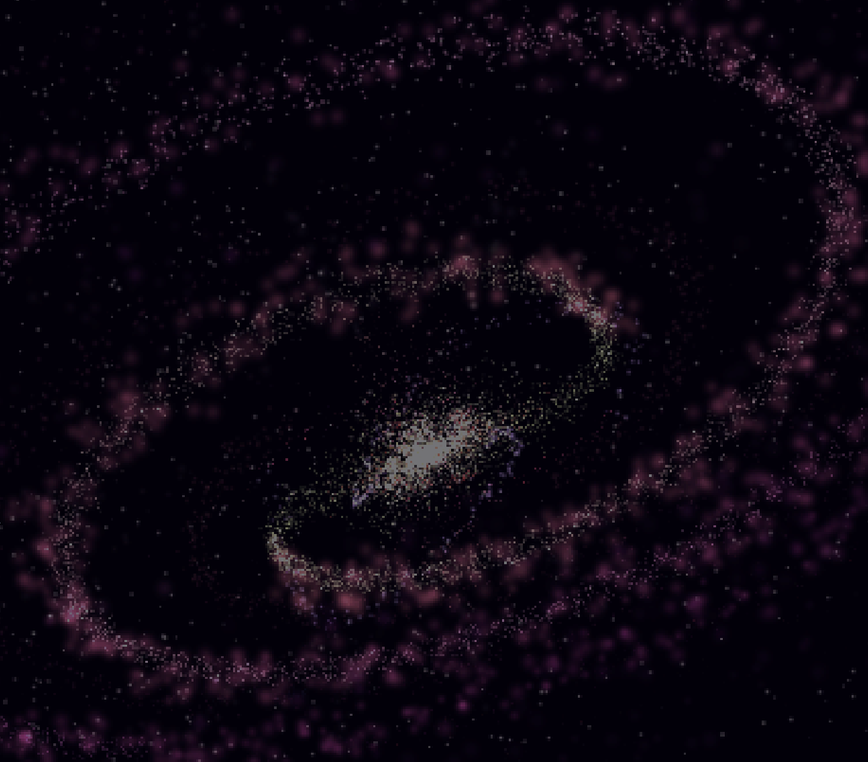
Reduce failures, ratchet the harness, and dogfood the result.

> **IMPORTANT**
> AI amplifies the system you give it. Build the evidence system before chasing velocity.

Build one layer lower—so every product can start higher

Name the contract. Create the witness. Let AI search. Keep the proof.

Then spend your ambition on what only your product can mean.



REPOSITORY

`github.com/odvcencio/gotreesitter`

BUILD WITH M31 LABS

`m31labs.dev/build`